BCSCHOOL

Empowering Digital Skills For The Jobs Of The Future

by

BC Soft
consulenza sistemi informativi

AiRED
Artificial Intelligence Ricerca e Sviluppo

JAVA JEE
-
Polimorfismo

TEACHER

Vincenza Ruvolo

vincenza.ruvolo@bcsoft.net

Sommario

- Polimorfismo
- Polimorfismo per metodi
- Overloading
- Varargs
- Overriding
- Polimorfismo per dati
- Tipo effettivo-Tipo referenziato
- Parametri polimorfi
- Collezioni eterogenee
- Casting di oggetti
- Invocazione metodi virtuali
- Polimorfismo e Interfacce

Polimorfismo

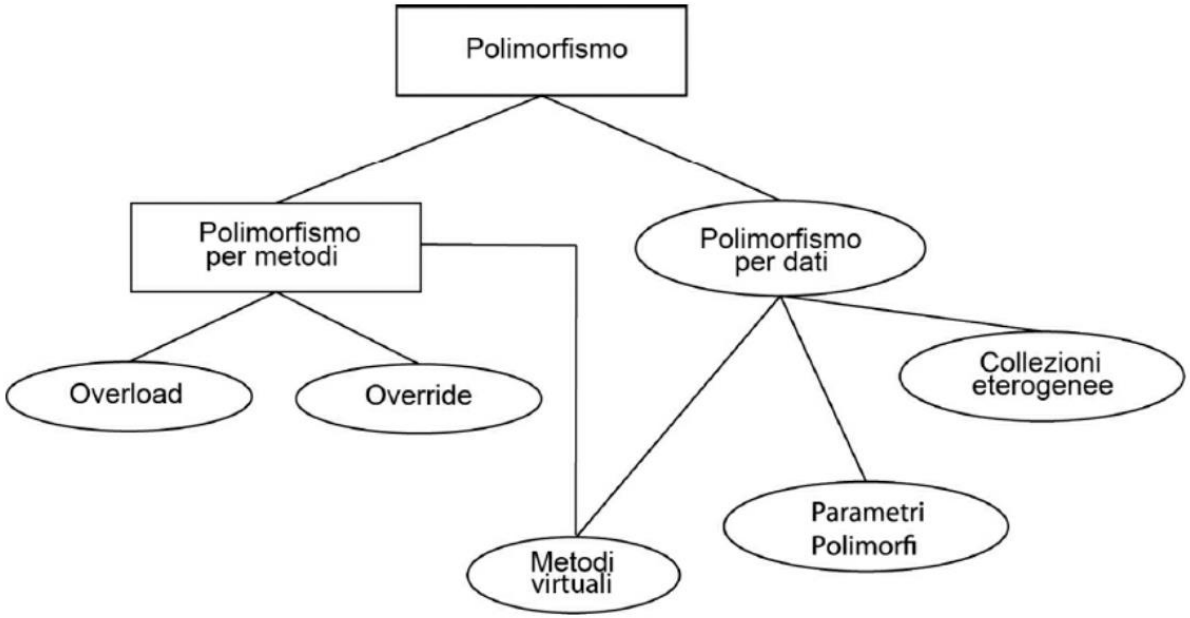
Il polimorfismo è un altro concetto che dalla realtà è stato importato nella programmazione ad oggetti, che permette di utilizzare un unico termine per riferirsi ad entità diverse.

Polimorfismo

Ad esempio, sia un telefono fisso sia uno smartphone consentono di telefonare, dato che entrambi i mezzi sono definibili come telefoni.

Polimorfismo

Dato il seguente schema, i rettangoli rappresentano i concetti, mentre gli ovali i concetti che hanno una reale implementazione in Java.



Polimorfismo

Convenzione per i reference

Un reference è come una variabile che contiene due informazioni rilevanti: l’indirizzo in memoria e l’intervallo di puntamento definito dalla relativa classe.

Polimorfismo

Si consideri il seguente esempio pratico: una classe che definisce un punto con coordinate (x,y).

Polimorfismo

```
package punto;
public class Punto {
  private int x;
  private int y;
  public void setX(int x) { this.x = x; }
  public void setY(int y) { this.y = y; }
  public int getX() { return x; }
  public int getY() { return y; }
}
```

Polimorfismo

Si istanzi un nuovo Punto.

```
Punto ogg = new Punto();
```

Polimorfismo

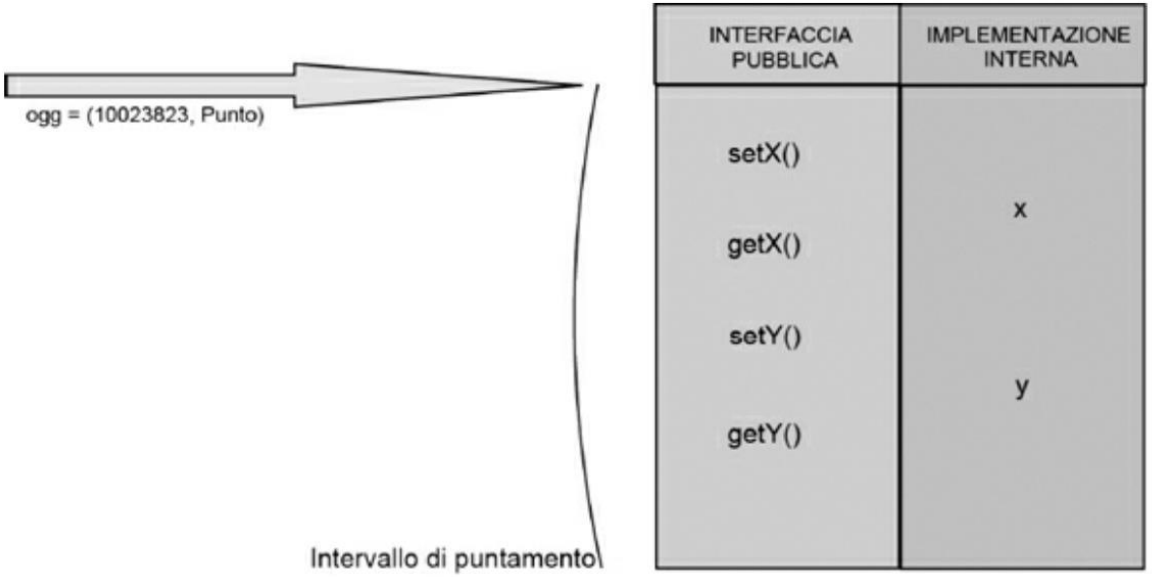
Viene creato un oggetto Punto che si suppone abbia come indirizzo un valore numerico, ad esempio 10023823, e come intervallo di puntamento Punto.

Polimorfismo

Il reference p accede all’interfaccia pubblica della classe Punto, ovvero a tutti i membri pubblici (setX(), setY(), getX(), getY()) dichiarati nella classe Punto tramite il reference p.

Polimorfismo

L'indirizzo invece farà puntare il reference ad una particolare area di memoria dove risiederà il particolare oggetto istanziato.



Polimorfismo per metodi

Overload

In un metodo, la coppia costituita dall’identificatore e dalla lista dei parametri è detta firma o segnatura del metodo.

Polimorfismo per metodi

In Java un metodo è univocamente determinato non solo dal suo identificatore ma anche dalla sua lista di parametri, cioè dalla sua firma.

Polimorfismo per metodi

Il metodo **Overload** dà la possibilità di utilizzare lo stesso nome per metodi diversi, metodi che concettualmente hanno la stessa funzionalità, ma soddisfano tale funzionalità in maniera differente, come è mostrato nel seguente script.

Polimorfismo per metodi

```
package aritmetica;
public class Aritmetica {
   public int somma(int a, int b) { return a + b; }
   public float somma(int a, float b) { return a + b; }
   public float somma(float a, int b) { return a + b; }
   public int somma(int a, int b, int c) { return a + b + c; }
   public double somma(int a, double b, float c) { return a + b + c; }
}
```

Polimorfismo per metodi

Il tipo di ritorno non fa parte della firma di un metodo, quindi, non ha importanza per l'implementazione dell'overload.

Polimorfismo per metodi

Varargs

È possibile utilizzare come argomenti dei metodi i cosiddetti varargs, in modo che un metodo accetti un numero non precisato di argomenti (compresa la possibilità di passare zero parametri), evitando così un eccessivo overload di metodi.

Polimorfismo per metodi

La sintassi dei varargs fa uso di ellissi (i puntini sospensivi "...”). Effettivamente i varargs, all’interno del metodo dove sono dichiarati, sono considerati a tutti gli effetti degli array.

Polimorfismo per metodi

Con i varargs potremmo creare un unico costruttore in questo modo:

```
package varargs;

public class ClienteVarargs {
   private String nome;
   private String indirizzo;
   private String numeroDiTelefono;

   public ClienteVarargs(String... par) {
     if(par != null && par.length > 0) {
            if(par.length >= 1 && par[0] != null) {this.nome = par[0];}
            if(par.length >= 2 && par[1] != null) {this.indirizzo = par[1];}
            if(par.length >= 3 && par[2] != null) {this.numero = par[2];}
           }
   }
   //...
}
```

Polimorfismo per metodi

Override

L’override (“riscrittura”) è il termine object oriented che viene utilizzato per descrivere la caratteristica che hanno le sottoclassi di ridefinire un metodo ereditato da una superclasse.

Polimorfismo per metodi

L’override deve rispettare delle regole per essere implementato:

1. Il metodo riscritto nella sottoclasse deve avere la stessa firma (nome e parametri) del metodo della superclasse;

Polimorfismo per metodi

2. Il tipo di ritorno del metodo della sottoclasse deve coincidere con quello del metodo che si sta riscrivendo, o deve essere di un tipo che estende il tipo di ritorno del metodo della superclasse;

Polimorfismo per metodi

3. Il metodo ridefinito nella sottoclasse non deve essere meno accessibile del metodo originale della superclasse.

Annotazione sull’override

Esiste nel package java.lang l’annotazione Override. Questa viene utilizzata per marcare i metodi che vogliono essere un override di metodi ereditati.

Annotazione sull'override

Per esempio:

```
package punto;

public class PuntoTridimensionale extends Punto {
private double ris;
   @Override
   public double distanzadallOrigine() {
      // ...
   }
}
```

L'utilizzo dell'annotazione richiede l'uso del carattere **@.**

Polimorfismo per dati

Il polimorfismo per dati permette essenzialmente di poter assegnare un reference di una superclasse ad un’istanza di una sottoclasse.

Polimorfismo per dati

Ad esempio considerando la classe PuntoTridimensionale, sottoclasse di Punto, sarà assolutamente legale scrivere:

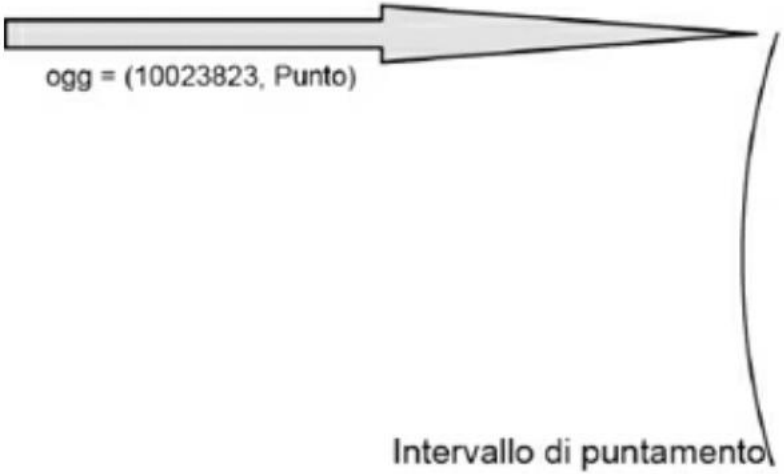
```
Punto ogg = new PuntoTridimensionale();
```

Polimorfismo per dati

L’interfaccia pubblica dell’oggetto creato (costituita dai metodi setX(), getX(), setY(), getY(), setZ(), getZ() e distanzaDallOrigine()) contiene l’interfaccia pubblica della classe Punto (costituita dai metodi setX(), getX(), setY(), getX() e distanzaDallOrigine()) e così il reference ogg “penserà” di puntare ad un oggetto Punto.

Polimorfismo per dati

Graficamente si avrà:



INTERFACCIA PUBBLICA	IMPLEMENTAZIONE INTERNA
setX()	
getX()	x
setY()	
getY()	
distanza...()	y
setZ()	
getZ()	

Polimorfismo per dati

Una limitazione di questo approccio è che un reference di una superclasse non potrà accedere ai campi dichiarati per la prima volta nella sottoclasse.

Parametri polimorfi

Si ricordi che i parametri in Java sono sempre passati per valore.

Parametri polimorfi

In un metodo, un parametro di tipo reference si dice **parametro polimorfo** quando può puntare ad un oggetto istanziato da una sottoclasse.

Parametri polimorfi

Sfruttando il polimorfismo per dati, un parametro di un metodo potrebbe puntare ad oggetti diversi.

È il caso del metodo println() che prende un parametro di tipo Object.

Collezioni eterogenee

Una **collezione eterogenea** è una collezione composta da oggetti diversi (ad esempio un array di Object che in realtà immagazzina altri tipi di oggetti).

Collezioni eterogenee

Un array dichiarato di Object potrebbe contenere ogni tipo di oggetto.

Collezioni eterogenee

Si consideri il seguente esempio allo scopo di intuire l'utilità di questi concetti. Per semplicità si ometteranno i metodi setter e getter nelle proprie classi.

Collezioni eterogenee

Immaginiamo di voler realizzare un sistema che stabilisca le paghe dei dipendenti di un’azienda, considerando le seguenti classi:

Collezioni eterogenee

```
package paghedipendenti;

public class Dipendente {
    private String nome;
    private int stipendio;
    private int matricola;
    private String dataDiNascita;
    private String dataDiAssunzione;

    //metodi setter e getter omessi
}
```

Collezioni eterogenee

```
package paghedipendenti;

public class Programmatore extends Dipendente {
    private String linguaggiConosciuti;
    private int anniDiEsperienza;

    //metodi setter e getter omessi
}
```

Collezioni eterogenee

```
package paghedipendenti;

public class Dirigente extends Dipendente {
   private String orarioDiLavoro;

   //metodi setter e getter omessi
}
```

```
package paghedipendenti;

public class AgenteDiVendita extends Dipendente {
   private String [] portafoglioClienti;
   private int provvigioni;

   //metodi setter e getter omessi
}
```

Collezioni eterogenee

Lo scopo è realizzare un metodo che stabilisca le paghe dei dipendenti. È possibile dichiarare una collezione eterogenea di dipendenti nel seguente modo:

```
package paghedipendenti;

public class Test {

  public static void main(String[] args) {
     Dipendente [] arr = new Dipendente [180];
     arr[0] = new Dirigente();
     arr[1] = new Programmatore();
     arr[2] = new AgenteDiVendita();
     // ...
   }
}
```

Collezioni eterogenee

Utilizzando il seguente operatore **instanceof**, è possibile verificare se l'oggetto è un'istanza del tipo specificato, cioè l'*operatore di confronto dei* tipi confronta l'istanza con il tipo.

Collezioni eterogenee

```
public static void pagaDipendente(Dipendente dip) {

   if (dip instanceof Programmatore) {
      dip.setStipendio(1500);
   }
   else if (dip instanceof Dirigente) {
      dip.setStipendio(3000);
   }
   else if (dip instanceof AgenteDiVendita) {
      dip.setStipendio(1000 + dip.getProvvigioni());
   }

   // ...

}
```

Collezioni eterogenee

Si inserisca questo metodo all'interno di un ciclo foreach (di 180 iterazioni), passandogli tutti gli elementi della collezione eterogenea.

```
// ...

for (Dipendente dipendente :  arr) {
   pagaDipendente(dipendente);
   // ...
}
```

Casting di oggetti

Esiste la possibilità di ristabilire la piena accessibilità all'oggetto tramite il meccanismo del casting di oggetti. Il casting di oggetti sfrutta una sintassi del tutto simile al casting tra dati primitivi:

```
if (dip instanceof Programmatore) {
  Programmatore pro = (Programmatore) dip;
  //...
}
```

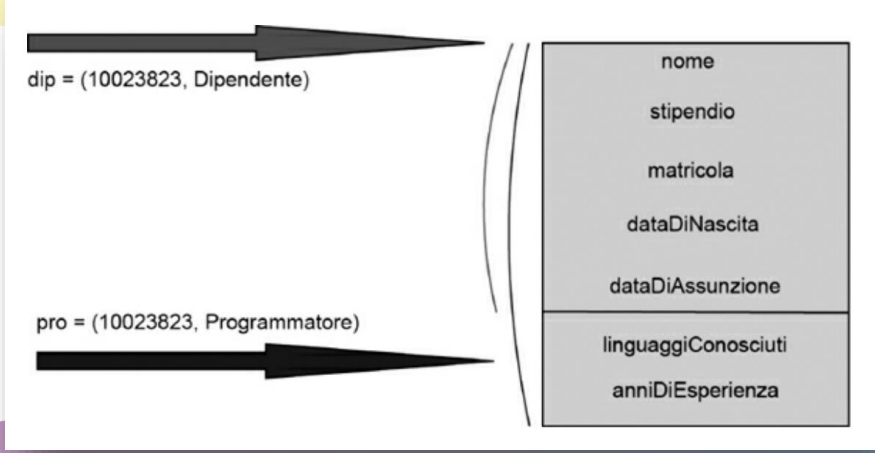
Casting di oggetti

È possibile accedere alla variabile anniDiEsperienza tramite la sintassi:

```
//...

if (pro.getAnniDiEsperienza() > 2)

//...
```



Casting di oggetti

L’accessibilità all’oggetto dipende dal fatto che i due reference abbiano lo stesso valore numerico (indirizzo) ma differente intervallo di puntamento.

Invocazione virtuale dei metodi

Un’invocazione ad un metodo m può definirsi virtuale quando m è definito in una classe A, ridefinito in una sottoclasse B (override) e invocato su un’istanza di B, tramite un reference di A (polimorfismo per dati).

Invocazione virtuale dei metodi

Quando s'invoca in maniera virtuale il metodo m, il compilatore "pensa" di invocare il metodo m della classe A (virtualmente). In realtà viene invocato il metodo ridefinito nella classe B.

Esempio pratico

Si considerino le seguenti classi:

```
package veicolo;

public abstract class Veicolo {

    public abstract void accelera();
    public abstract void decelera();

}
```

Esempio pratico

```
package veicolo;
public class Aereo extends Veicolo {
   public void decolla() { ... }
   public void atterra() { ... }
   @Override
   public void accelera() { ... }
   @Override
   public void decelera() { ... }

   //...
}
```

```
package veicolo;
public class Nave extends Veicolo {
  @Override
  public void accelera() { ... }

  @Override
  public void decelera() { ... }

  public void gettaAncora() { ... }

  //...

}
```

Esempio pratico

```
package veicolo;

public class Automobile extends Veicolo {
    @Override
    public void accelera() { ... }

    @Override
    public void decelera() { ... }

    public void innestaRetromarcia() { ... }

    //...

}
```

Esempio pratico

La superclasse Veicolo definisce i metodi accelera e decelera, che vengono poi ridefiniti in sottoclassi più specifiche quali Aereo, Automobile e Nave.

Esempio pratico

Si consideri la seguente classe, che fa uso dell’overload:

Esempio pratico

```
package veicolo;

public class Viaggiatore {

   public void viaggia(Automobile a) {

      a.accelera();
      //...

   }
   public void viaggia(Aereo a) {
      a.accelera();
      //...

   }
   public void viaggia(Nave n) {
      n.accelera();
      //...

   }
   //...

}
```

Esempio pratico

Nonostante l’overload rappresenti una soluzione notevole per la codifica della classe Viaggiatore, utilizzando un parametro polimorfo ed un metodo virtuale, la classe Viaggiatore potrebbe essere codificata in modo più compatto e funzionale:

Esempio pratico

```
public class Viaggiatore {
    public void viaggia(Veicolo v) { //parametro Polimorfo
        v.accelera(); //invocazione metodo virtuale
        //...
    }
    //...
}
```

Esempio pratico

Il seguente frammento di codice infatti, utilizza le precedenti classi:

```
// ...

Viaggiatore claudio = new Viaggiatore();
Automobile fiat500 = new Automobile();

// avremmo potuto istanziare anche una Nave o un Aereo
claudio.viaggia(fiat500);

// ...
```

Esempio pratico

Se si desidera introdurre un nuovo Veicolo (ad esempio la classe Bicicletta) basterebbe semplicemente codificarla.

Polimorfismo e interfacce

Il polimorfismo per dati funziona anche con le interfacce, cioè è possibile usare reference di interfacce per puntare ad oggetti che implementano queste interfacce.

Polimorfismo e interfacce

È stata definita l'interfaccia Volante in questo modo:

```
package veicolo;

public interface Volante {

    void atterra();
    void decolla();

}
```

Polimorfismo e interfacce

Ogni classe che deve astrarre un concetto di oggetto volante (come un aereo, un elicottero o un uccello) deve implementare l’interfaccia.

Polimorfismo e interfacce

La classe Aereo sarà:

```
package veicolo;

public class Aereo extends Veicolo implements Volante {

   // override del metodo di Volante
   @Override
   public void atterra() { }

   // override del metodo di Volante
   @Override
   public void decolla() { }

   // override del metodo di Veicolo
   @Override
   public void accelera() { }
   // override del metodo di Veicolo
   @Override
   public void decelera() { }

}
```

Polimorfismo e interfacce

È possibile creare parametri polimorfi per sfruttare l'interfaccia Volante:

```
package veicolo;

public class TorreDiControllo {
    public void autorizzaAtterraggio(Volante v) {
        v.atterra();
    }

    public void autorizzaDecollo(Volante v) {
        v.decolla();
    }
}
```

Domande e approfondimenti